

Rahnuma

रहनुमा رہنما

A working guide to the ILM, PANINI and Zistgah stack, from bytes and scripts to compilers, Pāṇini and quantum substrates

प्रकृति

	written	Devanagari hub	Romenagri	and back
Devanagari	प्रकृति	→ प्रकृति	→ prak_ri_ti	→ प्रकृति
Telugu	ప్రకృతి	→ प्रकृति	→ prak_ri_ti	→ ప్రకృతి
Bengali	প্রকৃতি	→ प्रकृति	→ prak_ri_ti	→ the same identity
Urdu, as usually written	ہندوی	→ हनदवी	→ _hana_dawee	→ हनदवी
Urdu, with zer and sukun	ہندوی	→ हिन्दवी	→ _hin_dawee	→ हिन्दवी

One identity, many scripts. Telugu and Bengali meet Devanagari in one hub, become the same Romenagri word, and come back exactly. Urdu meets the hub at what is written: without its short-vowel marks the word reads hanadawee; with zer and sukun it is hindawi. Every cell here was produced by the Hindawi tools while this guide was built.

Abhishek Choudhary, AyeAI. Version 2.0.0, 3 October 2026.

Rahnuma, the edition for the field: Natural sciences, mathematics and statistics

Contents

Part I: Orientation 2

1. How to use this guide	3
3. Start here: find your way in	4

Part IV: Foundations 6

17. Numbers inside the machine	7
22. Below the software: logic, chips, ternary and qubits	9
23. Quantum computing and PANINIq, with the mathematics	12

Appendices 16

D. For researchers in science, technology, engineering and mathematics	17
F. Formal definitions	19

Part I: Orientation

1. How to use this guide

In plain words. This guide explains a large body of work on computers, languages and intelligence. You can read it, run the programs on your own computer, or ask an AI to help you, and every example in it was really run.

This guide is for anyone who wants to use, check or extend the work gathered in the *zistgah*, *project-ilm* and *hindawiai* organisations on GitHub: students preparing for GATE, engineers who write compilers, firmware or chip layouts, linguists who work with scripts and sound, and researchers in quantum computing and AI. It asks for curiosity and a laptop, and nothing else.

There are four ways in.

- **Understand.** Part II gives the whole architecture first: where the work comes from, how the parts compose, each term with its nearest analogue and the difference, and the three Acts.
- **Read.** Parts IV and V explain the ideas the work rests on, with the mathematics written out: how numbers and letters sit in memory, what makes a transliteration reversible, how Pāṇini's grammar computes, how a compiler turns text into machine instructions, and how an oscillator network and a quantum circuit attack the same problem.
- **Run.** Part III gives the exact commands to set up Linux, Windows with WSL, or macOS, and to run every component of the estate on your own machine.
- **Ask.** Chapter 24 shows how to put any AI assistant to work on these repositories, and how to check what it tells you.

Conventions used throughout:

- Every example that could run while this guide was being built did run, and the text shows its real output, marked *Executed while this guide was built*. The package's own `gate`, `bash ops/verify.sh`, runs those examples again and fails if any output differs from what is printed here.
- Status words are used exactly. **Tested:** a test in the repository asserts it. **Executed:** it ran and its output is shown. **Written:** the code exists but has not been run here. **Mocked:** a stand-in replaces hardware that is not attached. **Planned:** described, not built.
- Everything said about the repositories was retrieved from the repositories themselves on 29 September 2026: their listings on GitHub, their `CITATION.cff` files and their READMEs. Where a repository does not record something, this guide says so instead of filling the gap.
- A dagger, †, marks a statement that follows the author's working notes and write-ups as supplied on 29 September and 3 October 2026, rather than the repositories.
- Sanskrit is transliterated in IAST. The spelling is British.

The work lives at github.com/zistgah, github.com/project-ilm and github.com/hindawiai, on the site zistgah.org, and in the Zenodo records listed in Chapter 12.

3. Start here: find your way in

In plain words. Four steps to find your part of the guide: who you are, your board or field, what you want to do, and your path with a PDF made for you.

This guide serves a school student and a senior researcher; a village health worker and an investor. Nobody needs all of it. Four steps find your part, on the site or with this page:

1. **Who you are.** A school student, a college student, a postgraduate or doctoral researcher; a teacher; an academic researcher; a working professional, someone in public service, policy or law, someone in community, NGO or village work, an artist, writer or maker; an industry partner or investor; or an AI agent working for a person.
2. **Your board or your field.** A school student chooses a board, ICSE, ISC, CBSE, IGCSE, IB, a state board or another; and a class. Everyone else chooses a field from the international standard classification of fields of study, ISCED-F 2013, the same one FAKIR's lattice uses. It places each subject where its own scholars would: political science is under social and behavioural sciences, not the humanities; law is under business, administration and law; linguistics is under languages.
3. **What you want to do.** Learn the basics, run the software, study or research, teach, contribute, or adopt, fund or invest.
4. **Your path and your edition.** The site lists your chapters in order, with reading times and the components to try first, and offers a PDF edition made for you. Every edition below is a printed selection from this one guide, so all of them stay in step.

Edition	PDF
School student	role-school.pdf
College student	role-college.pdf
Postgraduate or doctoral researcher	role-pg.pdf
Teacher or lecturer	role-teacher.pdf
Academic researcher	role-researcher.pdf
Working professional or engineer	role-professional.pdf
Public service, policy or law	role-public.pdf
Community, NGO or village work	role-community.pdf
Artist, writer or maker	role-creator.pdf
Industry partner, investor or funder	role-industry.pdf
Generic programmes and qualifications	field-00.pdf
Education	field-01.pdf
Arts and humanities	field-02.pdf
Social sciences, journalism and information	field-03.pdf

Edition	PDF
Business, administration and law	field-04.pdf
Natural sciences, mathematics and statistics	field-05.pdf
Information and Communication Technologies (ICTs)	field-06.pdf
Engineering, manufacturing and construction	field-07.pdf
Agriculture, forestry, fisheries and veterinary	field-08.pdf
Health and welfare	field-09.pdf
Services	field-10.pdf

The first public version of this guide, written for advanced researchers and senior professionals, stays as it was at [version 1](#).

Part IV: Foundations

17. Numbers inside the machine

In plain words. How a computer stores numbers using only ones and zeros, including negative numbers and fractions.

A processor holds voltages that stand for bits. Everything else on this page, including the hexadecimal digits, is notation for people.

Positional notation

A number written with digits d_i in base b is

$$n = \sum_{i=0}^{k-1} d_i b^i, \quad 0 \leq d_i < b.$$

So $42 = 101010_2 = 2A_{16}$: the letters A to F are simply names for the values ten to fifteen, chosen so that one hexadecimal digit covers exactly four bits. The machine never sees the letter A, any more than it sees the letters of a keyword in a program.

Signed integers: two's complement

With n bits $b_{n-1} \dots b_0$, the value is

$$v = -b_{n-1} 2^{n-1} + \sum_{i=0}^{n-2} b_i 2^i, \quad -2^{n-1} \leq v \leq 2^{n-1} - 1.$$

Negative x is therefore stored as $2^n - |x|$. The payoff is that one adder serves signed and unsigned arithmetic alike, which is why every processor in use works this way.

Reals: IEEE 754

A double-precision number has a sign bit s , eleven exponent bits e and fifty-two fraction bits f , and for normal numbers

$$v = (-1)^s \times (1.f)_2 \times 2^{e-1023}.$$

One tenth has no finite binary expansion, so it is stored rounded, and sums of rounded values are rounded again.

Byte order and bit order

A 32-bit value occupies four bytes. A little-endian machine (x86, and ARM as usually configured) stores the least significant byte at the lowest address; a big-endian machine stores the most significant byte first, and network protocols use big-endian, which is why C has `htonl` and `ntohl`. That is endianness, and it concerns bytes.

Bit numbering is a separate convention: whether bit 0 is the least or the most significant bit. The hexadecimal digit E is **1110** when written most significant bit first and **0111** when written least significant bit first, and a serial line such as a UART sends the least significant bit first. Both conventions are representations. The circuit has only the wires.

Executed while this guide was built: `python3 examples/numbers_demo.py`

Integers and reals

```
0x0A0B0C0D little-endian bytes: 0d 0c 0b 0a    big-endian: 0a 0b 0c 0d
-5 in 8-bit two's complement: 11111011    (256 - 5 = 251)
0.1 as an IEEE 754 double: 3fb999999999999a = 0x1.999999999999ap-4
0.1 + 0.2 == 0.3 is False; the sum is 0.30000000000000004
```

These are the facts the GATE syllabus asks for under number representation and computer arithmetic, fixed and floating point (Chapter 26).

22. Below the software: logic, chips, ternary and qubits

In plain words. What happens inside chips: switches, circuits, three-valued logic, and the very cold machines that hold qubits.

From logic to CMOS

Every digital function reduces to Boolean algebra, and NAND alone is enough to build all of it. In CMOS, a network of p-type transistors pulls the output up and a dual network of n-type transistors pulls it down, so that in a settled state no current flows from supply to ground. Power is spent mainly in switching, $P \approx \alpha C V^2 f$, with α the fraction of nodes switching per cycle, C the capacitance switched, V the supply voltage and f the clock frequency; the V^2 is why every generation of chips has lowered its voltage.

How a chip is designed

1. Describe the hardware at register-transfer level in Verilog, VHDL or SystemVerilog, or generate it from Chisel or Amaranth.
2. Simulate it, with Icarus Verilog or Verilator.
3. Synthesise it into a netlist of standard cells, with Yosys.
4. Floorplan, place, build the clock tree and route, with OpenROAD.
5. Check design rules and compare layout against schematic, with Magic, KLayout and Netgen.
6. Write the layout as GDSII or OASIS and send it to the foundry: the tapeout.
7. Fabricate, test and package.

Open process design kits let anyone take this path to real silicon: SkyWater SKY130 (130 nm), GlobalFoundries GF180MCU (180 nm) and IHP SG13G2 (130 nm, SiGe BiCMOS). [zistgah/jugaad28](https://www.zistgah.com/jugaad28) catalogues integrated circuits taped out on 28 nm, with evidence-gated descriptors and a low-cost open laboratory. A hardware description language is a language like any other: its front end can be given native-language keywords by the three-axis method of Chapter 21, and what reaches the synthesiser is a netlist, not words.

Instruction sets and the boot chain

The instruction set is the contract between software and hardware. RISC-V is an open one: a small base, RV32I or RV64I, with standard extensions for multiplication (M), atomics (A), floating point (F and D), compressed instructions (C) and vectors (V), which anyone may implement without a licence fee. An open instruction set, open design tools and reproducible toolchains carry the argument of Chapter 21 down to the silicon.

A computer starts through a chain of programs: a boot ROM, the firmware (BIOS or UEFI), a boot loader such as GRUB, the kernel, the first user process such as systemd, and then everything else. Each is a program in some language, and each can be given a native-language front end without changing what the hardware executes; the hindawiai organisation keeps a fork of the Linux kernel tree, [hinlin](https://github.com/hindawiai/linux), for exactly this work.

Balanced ternary: Zamin and PRATIK

Balanced ternary uses the digits -1 , 0 and $+1$, written T, 0 and 1:

$$n = \sum_i t_i 3^i, \quad t_i \in \{-1, 0, +1\}.$$

It needs no sign: a number is negated by flipping every digit, and rounding is simply truncation. It is also the most economical integer base. Representing numbers up to N in base b takes about $\log_b N$ digits of b states each, a cost proportional to

$$b \log_b N = \frac{b}{\ln b} \ln N,$$

which is least at $b = e \approx 2.718$; among whole numbers, 3 wins over 2 and 4. The Setun computer, built at Moscow State University in 1958, ran on balanced ternary.

[zistgah/zamin](#) is the physical realisation of the PRATIK balanced-ternary substrate for the PEDLER event algebra. Its poised zero is a physical high-impedance ground, so that the electrical ground reference and the computational zero are the same node, and its spawning operator creates new physical dimensions on a memristive three-dimensional crossbar. The repository holds SPICE cells, an interactive circuit simulator and a paper; [zistgah/pratik_core_mvp](#) is the kernel in C++ and CUDA.

Executed while this guide was built: `python3 examples/ternary.py`

```
5 =      1TT   negated by flipping every digit:      T11 = -5
-5 =      T11   negated by flipping every digit:      1TT = 5
13 =      111   negated by flipping every digit:      TTT = -13
40 =      1111  negated by flipping every digit:      TTTT = -40
-121 =     TTTT  negated by flipping every digit:     11111 = 121
2026 =    10T10001 negated by flipping every digit:    T01T000T = -2026
```

```
radix economy b/ln b for b = 2.000: 2.8854
radix economy b/ln b for b = 2.718: 2.7183
radix economy b/ln b for b = 3.000: 2.7307
radix economy b/ln b for b = 4.000: 2.8854
radix economy b/ln b for b = 10.000: 4.3429
```

Superconducting qubits

A Josephson junction is two superconductors separated by a thin insulator, usually aluminium, aluminium oxide and aluminium. Its current and voltage obey

$$I = I_c \sin \varphi, \quad V = \frac{\hbar}{2e} \frac{d\varphi}{dt},$$

where φ is the phase difference across it. It behaves as a nonlinear inductor, and the nonlinearity is what makes the lowest two energy levels of a circuit usable as a qubit. The transmon, the most common design, has the Hamiltonian

$$H = 4E_C (\hat{n} - n_g)^2 - E_J \cos \hat{\varphi},$$

and is operated with E_J/E_C around 50, which makes it nearly insensitive to stray charge (Koch and colleagues, 2007). Its transition frequency lies around 4 to 8 GHz.

The cold follows from that frequency. Thermal energy has to be far below the energy of one microwave photon at the qubit frequency. At 10 mK, which is -273.14°C , $k_B T/h$ is about 0.21 GHz, well below 5 GHz; at the 4 K of liquid helium it would be about 83 GHz, and the qubit would be swamped by thermal excitation. Hence the dilution refrigerator, the large gold-coloured structure in photographs of quantum computers; the processor itself is a small chip at its bottom.

That chip is an integrated circuit: aluminium or niobium films on silicon or sapphire, patterned by lithography from a layout file like any other chip, through processes related to, but distinct from, CMOS logic processes. Qiskit Metal is IBM's open tool for laying out and analysing such chips. Other ways to build qubits are trapped ions, neutral atoms, photons and spins in silicon; nitrogen-vacancy centres in diamond keep their spin coherence at room temperature and are used today as sensors.

PANINIq's oscillator substrate, Chapter 23, is different in kind: a classical network of coupled oscillators that can be built from ordinary electronics and behaves as an Ising machine. Its hardware bridge is mocked today; its mathematics is not.

23. Quantum computing and PANINIq, with the mathematics

In plain words. How quantum computers work, and how a network of swinging oscillators can solve the same kind of puzzle.

States, gates and measurement

A qubit's state is a unit vector in $\mathbb{C}^2$,

$$|\psi\rangle = \alpha|0\rangle + \beta|1\rangle, \quad |\alpha|^2 + |\beta|^2 = 1,$$

which, up to a global phase, is a point on the Bloch sphere: $|\psi\rangle = \cos \frac{\theta}{2} |0\rangle + e^{i\phi} \sin \frac{\theta}{2} |1\rangle$. The state of n qubits lives in the tensor product of n copies, a space of 2^n complex amplitudes. Gates are unitary matrices:

$$X = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}, \quad Z = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}, \quad H = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}, \quad \text{CNOT} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{pmatrix}.$$

A Hadamard followed by a CNOT entangles two qubits: $\text{CNOT}(H \otimes I)|00\rangle = \frac{1}{\sqrt{2}}(|00\rangle + |11\rangle)$. Measuring in the computational basis gives outcome x with probability $|\langle x|\psi\rangle|^2$, the Born rule. Between measurements the state evolves under a Hamiltonian H by $i\hbar \frac{d}{dt}|\psi\rangle = H|\psi\rangle$, so that, in units with $\hbar = 1$,

$$|\psi(t)\rangle = e^{-iHt} |\psi(0)\rangle.$$

Simulating n qubits exactly means storing 2^n amplitudes of 16 bytes each: 30 qubits need 16 GiB. That is why a laptop simulates up to about thirty qubits, and why the encoding of a problem into qubits matters so much.

Ising, QUBO and Max-Cut

An Ising problem assigns spins $s_i \in \{-1, +1\}$ to minimise

$$E(s) = - \sum_{i < j} J_{ij} s_i s_j - \sum_i h_i s_i.$$

With $x_i = (1 - s_i)/2 \in \{0, 1\}$ the same problem becomes a quadratic unconstrained binary optimisation (QUBO). Max-Cut asks for the split of a weighted graph's vertices into two sides that maximises the weight of the edges crossing between them:

$$C(s) = \sum_{(i,j) \in E} w_{ij} \frac{1 - s_i s_j}{2}.$$

Each term is w_{ij} when the edge is cut and 0 when it is not, so maximising C is minimising $\sum w_{ij}s_i s_j$, the energy of an anti-ferromagnet with $J_{ij} = -w_{ij}$. Max-Cut is NP-hard; the best classical guarantee in polynomial time, Goemans and Williamson's semidefinite relaxation with random rounding, reaches 0.878 of the optimum.

QAOA

The Quantum Approximate Optimisation Algorithm prepares

$$|\gamma, \beta\rangle = \prod_{k=1}^p e^{-i\beta_k H_M} e^{-i\gamma_k H_C} |+\rangle^{\otimes n}, \quad H_C = \sum_{(i,j) \in E} w_{ij} \frac{1 - Z_i Z_j}{2}, \quad H_M = \sum_i X_i,$$

and a classical optimiser adjusts the $2p$ angles to maximise $\langle \gamma, \beta | H_C | \gamma, \beta \rangle$; measuring the final state gives candidate cuts. With $p = 1$ on 3-regular graphs, the expected cut is at least 0.6924 of the optimum (Farhi, Goldstone and Gutmann, 2014). QAOA uses one qubit per vertex, so exact simulation costs 2^N amplitudes; PANINIq's QAOA baseline therefore stops at $N \leq 12$ and uses the COBYLA optimiser.

The continuous-time quantum walk, compactly encoded

A continuous-time quantum walk (Farhi and Gutmann, 1998) evolves a state over the vertices of a graph under a Hamiltonian built from the graph: $|\psi(t)\rangle = e^{-iHt}|\psi(0)\rangle$. PANINIq encodes the N vertices as the basis states of $n = \lceil \log_2 N \rceil$ qubits, a binary address, so its Hilbert space has $2^n < 2N$ dimensions rather than 2^N . Its Hamiltonian is the coupling matrix placed in the top-left block of a $2^n \times 2^n$ zero matrix, and the evolution is one arbitrary unitary gate:

```
# pedler/quantum_validator.py in zistgah/paniniq, commit 82a5cf2 (excerpt)
num_qubits = max(1, int(np.ceil(np.log2(max(N, 2)))))
dim = 2 ** num_qubits
H = np.zeros((dim, dim), dtype=complex)
H[:N, :N] = K_matrix
...
U_t = expm(-1j * H * evolution_time)
qc.unitary(Operator(U_t), list(range(num_qubits)), label="CTQW(K,t)")
```

The test suite checks that the circuit's statevector equals e^{-iHt} applied directly. A general unitary on n qubits compiles into $O(4^n)$ elementary gates, so this is a construction for validating the classical dynamics, not an efficient circuit for hardware; the repository says the same.

The Jelly Substrate: coupled phase oscillators

The substrate is a network of phase oscillators obeying the Kuramoto equations, with coupling that decays with distance:

$$\frac{d\theta_i}{dt} = \omega_i + \sum_j K_{ij} \sin(\theta_j - \theta_i), \quad K_{ij} = K_0 e^{-\alpha r_{ij}}, \quad K_{ii} = 0.$$

```
# pedler/substrate.py in zistgah/paniniq, commit 82a5cf2 (excerpt)
def kuramoto_rhs(self, t, theta):
    phase_diffs = theta[None, :] - theta[:, None]
    return self.omegas + np.sum(self.K * np.sin(phase_diffs), axis=1)
```

How synchronised the network is shows in the order parameter,

$$R e^{i\psi} = \frac{1}{N} \sum_{j=1}^N e^{i\theta_j}, \quad 0 \leq R \leq 1,$$

near 1 when the phases lock together and near 0 when they are scattered. For identical all-to-all coupling K/N , each oscillator feels only the mean field, $\dot{\theta}_i = \omega_i + KR \sin(\psi - \theta_i)$, and synchrony appears once K exceeds $K_c = 2/(\pi g(0))$, where g is the symmetric, single-peaked distribution of natural frequencies; for a Lorentzian of half-width γ , $K_c = 2\gamma$.

Oscillators as an Ising machine

Coupled oscillators can minimise an Ising energy directly (Wang and Roychowdhury, 2019). With an added second-harmonic injection of strength K_s ,

$$\frac{d\theta_i}{dt} = -K \sum_j J_{ij} \sin(\theta_i - \theta_j) - K_s \sin(2\theta_i),$$

the function

$$\mathcal{E}(\theta) = -K \sum_{i,j} J_{ij} \cos(\theta_i - \theta_j) - K_s \sum_i \cos(2\theta_i)$$

never increases along the motion, so the network settles into its minima. The injection pushes each phase to 0 or π , where $\cos(\theta_i - \theta_j) = s_i s_j$ with $s_i = \cos \theta_i = \pm 1$, and $\mathcal{E}$ becomes the Ising energy. PANINIq's Max-Cut solver uses the simplest form of this: it sets the coupling to minus the adjacency, so that neighbours repel in phase, relaxes, and reads each spin as the sign of $\cos \theta_i$. Its tests check the result against brute force on six graphs: never above the optimum, and exact on the bipartite ones.

```
# pedler/maxcut.py in zistgah/paniniq, commit 82a5cf2 (excerpt)
sub.K = -1.0 * adjacency          # connected nodes repel in phase
...
spins = np.sign(np.cos(final_phases))
```

The PEDLER automaton on the substrate

PANINIq implements PEDLER as a six-tuple $P = (I, G, U, S, F, *)$. U is the stream of point events, vectors in $\mathbb{R}^3$. S is the set of states, one per oscillator, and F maps each state to a point in $\mathbb{R}^3$. G is the resolution-match kernel, $\exp(-\gamma \|F(v_i) - r_t\|)$, the similarity between an event r_t and each state. I is the inclination field over the states, a historical prior plus the weighted resolution match, computed afresh for each event. The operator $*$ governs growth: when an event is presented and the relaxed network fails to lock, its order parameter falling below the novelty threshold, a new state is spawned;

on a separate schedule, faded states are pruned and strongly locked ones consolidated. “Inclination field”, “resolution match” and “point event” are the framework’s own terms; the code implements the equations as given and claims nothing more about cognition. The repository also records one open question plainly: PEDLER’s canonical inclination is signed, lives on edges and lies in $[-1, +1]$, while this code’s is non-negative and lives on nodes, and which one the substrate should carry is the author’s decision.

What is tested, and what is open

From the repository’s own status table: the Jelly Substrate, the PEDLER engine, the Max-Cut solvers, the frame packing of the hardware bridge, the Hamiltonian and unitary mathematics, the Qiskit circuit and QAOA are **tested**; the PyTorch substrate is **written** and not executed; the hardware port is **mocked**; a Duffing extension with amplitude as well as phase, a real firmware protocol, and the larger system running from brain signals through symbolic orchestration to a physical instruction set and manufacturing are **planned**. The run in Chapter 14 shows all of it: states spawning as the order parameter drops, a three-qubit walk whose statevector sums to one, and the oscillator, brute force and QAOA agreeing on a cut of 4.

Exercises

1. Compute $H|0\rangle$, $H|1\rangle$ and HZH .
2. Show that $\frac{1}{2}(1 - \mathbf{s}_i \mathbf{s}_j)$ is 1 exactly when an edge is cut.
3. For a graph with 12 vertices, how many qubits do the walk and QAOA each need, and how many amplitudes does each simulation store?
4. Differentiate $\mathcal{E}$ along the motion and show that $d\mathcal{E}/dt \leq 0$ when $\mathbf{J}$ is symmetric.
5. In PANINIq, change the novelty threshold and predict, before running, how the number of spawned states changes.

Appendices

D. For researchers in science, technology, engineering and mathematics

In plain words. A guide for researchers in science, technology, engineering and mathematics: what can be read, run and tested now, and what is still open.

Each field below lists what in the estate can be read, run and tested now, and what is open. Status words keep their meaning from Chapter 1: tested, executed, written, mocked, planned.

Computer science

Programming languages and compilers. Hindawi's composed transducers over unmodified host toolchains; the Romenagri symbol bridge through ELF, DWARF and GDB; the PANINI construct model, with 39 constructs in 27 languages and 216 decorators across 8 host languages; the bootstrap and its self-hosting fixed point (Chapters 21 and 6). Executed here. **Open:** `std2hin` doubles every underscore, so vowel-initial names do not round-trip back from C; the guru shaili emits no `#line` directives; the YACC shaili awaits a fresh conformance run.

Systems. The hinlin kernel tree and the mass conversion of real C sources, including kernel trace code, into Hindi and back (`hindawiai/chintamani`, `Romenagri/mass_hindawi`).

AI and machine learning. The cyclers and GENIE as AI-agnostic, human-inspected method (Chapter 8); Romenagri as a tokenisation substrate, where bytes fall 1.76 times and the alphabet from 51 to 24 symbols without loss, but token counts did not fall at matched merges, so the predicted gain is still to be shown; COPA's Authority Projection Index as a measurable property of a model's behaviour.

Software engineering. Executable contracts and gates in every repository; the provenance chain, Tok DOI, spiguard, Candor and Misty DOI; VGC; the component cards of Chapter 15, each the result of a real run.

Data. ILM's registry of 7,867 languages and 226 scripts; 13,668 keyword standards; the corpora in the Romenagri tree.

Electrical and electronic engineering

The open chip-design flow and the 28 nm tape-out catalogue, jugaad28, with its evidence-gated descriptors (Chapter 22); Zamin's ternary cells in SPICE, whose testbench needs attention on ngspice 42 (Chapter 15); the PRATIK kernel, whose CPU build passes and whose CUDA build needs an NVIDIA toolchain; device control on ESP32, zasab, built but not yet pushed.

Physics

QEDLER, the event-hypergraph research programme for physics, self-declared as a framework with open problems; oscillator Ising machines and Kuramoto dynamics in PANINIq, with their Lyapunov function (Chapter 23); superconducting qubits and the physics of the cold (Chapter 22).

Mathematics

Coding theory for reversible transliteration: Kraft and McMillan, and Sardinas and Patterson (Chapter 19); formal languages and rewriting in Pāṇini's grammar, with pratyāhāras as interval classes over an ordered list (Chapter 20); graph theory in the typed hypergraph and in Max-Cut; dynamical systems in the order parameter and in the oscillators' energy; the formal tuples of the CEM kernel, $\mathcal{E} = (E, \mathcal{C}, \Pi, W)$, and of the AyeAI Triad.

Biology and medicine

PANINIB, the biological realization arm, which lowers intent to sequence IR and claims no wet-laboratory result; CEMb, operated through Interglial Healthcare; the Quantum Neuromorphic BCI programme, with brain-to-text, cognitive prosthetics and intelligent rehabilitation as directions needing clinical collaboration; VIKRAM's Health section and its tools for an accessible virtual hospital; the earlier HMSEI wearable and the Dr Rho telepresence platform.

Cognitive science and neuroscience

PEDLER's two branches, through LVF and MLCNE to the cognitive enablement modules, and through NI2A2 to ANGEL; its primitives; the brain-computer interface work of Act III.

Astronomy and the earth and space sciences

CHAKRA, the offline observatory and calendars, whose JavaScript and C99 versions produce byte-identical output; Jyotish, derived from it; the habitats, with a lander embodiment on the Moon and a quadroter on Mars inside the shared dome.

Robotics and manufacturing

PANINIphy, the physical realization arm, which compiles physical intent and chooses parts by force, stroke and voltage; pench, the embodied cyler; † the modular self-reconfiguring robots of the notes.

F. Formal definitions

In plain words. The same ideas written as mathematics, for readers who want the formal version.

This appendix states in symbols what the chapters state in words. Where the record gives a formal object, the PEDLER six-tuple, the CEM kernel or the Triad's tuples, it is reproduced as recorded. Where the record gives only words, the formalisation is this guide's reading, and is said to be.

The estate as a typed graph

The estate is a typed graph

$$H = (V, E, \tau_V, \tau_E), \quad E \subseteq V \times V \times R,$$

where V is the set of components, R the set of relations (projects, script layer, realized by, front end, mounts and the rest), $\tau_V : V \rightarrow \mathcal{F}$ assigns each component its family, and τ_E each edge its relation. The core is described as a typed hypergraph, in which one edge may join several components at once: $E \subseteq 2^V \times R$. A traversal is a walk $v_0 \xrightarrow{r_1} v_1 \xrightarrow{r_2} \dots \xrightarrow{r_k} v_k$; every sequence printed in this guide, and every reading path of the reader views, is one.

In this guide's reading, a projection $p \in \{\text{Kaivalyik, Zistgah, Cosmopolis}\}$ is a map $\pi_p : H \rightarrow H_p$ that keeps every component's stem and every relation and changes names: $\text{stem}(\pi_p(v)) = \text{stem}(v)$ and $(u, v, r) \in E \Rightarrow (\pi_p(u), \pi_p(v), r) \in E_p$. The Acts are a separate, partial labelling $\alpha : V \rightarrow \{\text{I, I|II, II, II|III, III}\}$, with PANINI in every Act.

Romenagri as a code

Let Σ be the Devanagari alphabet and $\Gamma = \{\mathbf{a}, \dots, \mathbf{z}, _ \}$, a subset of the alphabet of C identifiers. Romenagri's compiler form is a map

$$T : W \rightarrow \Gamma^*, \quad W \subseteq \Sigma^*,$$

injective on well-formed text W , with an inverse T^{-1} on $T(W)$. Text outside W is first brought to its canonical form by $c : \Sigma^* \rightarrow W$, so that for every string

$$T^{-1}(T(s)) = c(s), \quad c(s) = s \text{ whenever } s \in W.$$

On the 984 words of the Hindi corpus, $T^{-1}T(w) = w$ for 983; the one exception is a misspelling in the corpus.

For a Brahmi script σ whose Unicode block follows the ISCII layout from base b_σ , the hub map is $h_\sigma(x) = x - b_\sigma + \mathbf{0x900}$ on the aligned letters, a bijection onto the corresponding Devanagari letters; `flatten_uni_dev` applies it by table. A word in script σ therefore has the identity $T(h_\sigma(w))$, and returns by $h_\sigma^{-1}(T^{-1}(\cdot))$.

Urdu meets the hub through $\varphi : \Sigma_{\text{ur}}^* \rightarrow \Sigma^*$, which is many-to-one on unmarked text: the fibre $\varphi^{-1}(w)$ over a hub word can hold several spoken readings, because the short vowels are not written. With tashkil, the restriction of φ to marked text distinguishes them: unmarked ہندوی maps to हन्दवी , and marked ہندوی to हिन्दवी . In the corpus, 1 word in 794 carries a vowel mark.

The symbol bridge

For every name n in a program, the toolchain sees $\text{sym}(n) = T(n) \in \Gamma^*$. Each stage t_k between source and debugger (compiler; assembler; linker, ELF writer, DWARF writer; debugger) carries identifiers over Γ unchanged: $t_k(\text{sym}(n)) = \text{sym}(n)$. Hence

$$T^{-1}(t_m \circ \dots \circ t_1(\text{sym}(n))) = c(n),$$

which is the name as written. Chapter 21 runs this for every name of a program.

The construct model

Let $\mathcal{C}$ be the constructs (39), $\mathcal{L}$ the languages (27) and $\mathcal{H}$ the host languages (8). Each language ℓ has a keyword map $\kappa_\ell : \mathcal{C} \rightarrow W_\ell$, and each host h a realisation $\rho_h : \mathcal{C} \rightarrow W_h$, partial because not every host has every construct. Translation from ℓ to h is

$$\tau_{\ell \rightarrow h} = \rho_h \circ \kappa_\ell^{-1},$$

defined when κ_ℓ is injective, which is the reversibility check ILM runs on every keyword table. Direct pairings of a language with a host would number $|\mathcal{L}| \cdot |\mathcal{H}| = 27 \times 8 = 216$ tables; the construct model needs $|\mathcal{L}| + |\mathcal{H}| = 35$.

PEDLER, as PANINIq implements it

PANINIq implements PEDLER as the six-tuple

$$P = (I, G, U, S, F, *),$$

with events $U \subset \mathbb{R}^3$, states S , a feature map $F : S \rightarrow \mathbb{R}^3$, and the resolution kernel

$$G(v, r) = e^{-\gamma \|F(v) - r\|}.$$

The inclination of a state is its historical prior plus the weighted match, $I(v) = \eta(v) + w G(v, r)$. The operator $*$ grows and prunes the state set: a new state is spawned when the relaxed network's order parameter falls below a novelty threshold, $R < \theta$. The Act is the transition $A(I, S_t) \rightarrow S_{t+1}$. In this guide's reading, Natural Justice is an admissibility predicate on the passage from intent to Act: a transition fires only if $\text{NJ}(\text{intent}, S_t)$ holds.

Kernels and closures, as recorded

The record gives the CEM kernel and the AyeAI Triad's closure as tuples, and they are reproduced here as given; the expansions of their letters are not part of the retrieved record:

$$\mathcal{E} = (E, \mathcal{C}, \Pi, W), \quad \text{AyeAM} = \langle S, R, C \rangle, \quad \text{AyeAI} = \langle M, I, G \rangle, \quad \text{AyeCNSe} = \langle T, Ch, \Sigma \rangle.$$

Gates and evidence

A gate is a function $g : \mathcal{A} \rightarrow \{\text{pass}, \text{fail}, \text{unjudged}\}$ on artifacts, computed by running it. An issue is done exactly when

$$g(a) = \text{pass} \wedge \text{merged}(a) \wedge (\text{published}(a) \Rightarrow \text{receipt}(a)).$$

VGC's rule is that a worker's report $r(a)$ is not an argument of g : the gate is computed from the artifact, never read from the claim.

Provenance

A repository's manifest is the list $m = [(p_i, H(f_i))]_i$ of every tracked path with the SHA-256 of its file. Sealing commits the digest $d = H(m)$ to OpenTimestamps, whose calendars aggregate many digests in a Merkle tree anchored in a Bitcoin block; the proof for one digest among N is a path of length $\lceil \log_2 N \rceil$, and verifying it recomputes the root. The four systems are ordered by implication:

$$\text{mint}(a) \Rightarrow \text{attest}(a) \Rightarrow \text{clear}(a) \Rightarrow \text{seal}(a).$$

The seed space and the reading paths

FAKIR seeds a task from a domain node, an AGI layer and a language: $S = D \times L \times \Lambda$, with $|D| = 1,505$ nodes across ISIC, ISCO and ISCED, $|L| = 10$ layers and $|\Lambda| = 7,867$ languages, so

$$|S| = 1,505 \times 10 \times 7,867 = 118,398,350.$$

A cross-domain task is seeded from a set of nodes, a subset of D , rather than one.

A reader view is a sequence of chapters $(c_1, \dots, c_k)$, a walk on the chapters. A chapter of w words is estimated at $t(c) = \max(1, \text{round}(w/200))$ minutes, and a path at $\sum_j t(c_j)$.

© 1993–2026 Abhishek Choudhary. All rights reserved. AyeAI. Text CC BY-SA 4.0; scripts GPL-3.0-or-later. Version 2.0.0, 3 October 2026.